

Digital Forensics

The Initial Assessment

The initial assessment is the **first critical phase** of a digital forensic investigation.

Key Objectives:

Objective	Description
Identify	What devices/media are present
Preserve	Prevent alteration of evidence
Document	Record the scene before touching anything
Evaluate	Determine scope of investigation

Key Questions to Ask:

- What **type of crime** has occurred?
- What **devices** are involved?
- Is the device **powered on or off**?
- Is there **network connectivity**?
- What is the **legal authority** to seize?

*Critical Rule: The first responder's actions can **make or break** the entire investigation*

Incident Notification Checklist

A structured checklist ensures **nothing is missed** at the scene.

Checklist Components:

Date and Time of incident reported
Who reported the incident

Nature of the incident
Systems/devices affected
Has the scene been secured?
Are there witnesses present?
Has anything been touched/moved?
Photography/video of scene taken?
Chain of custody documentation started?
Legal authority confirmed?
Senior officer/manager notified?
Is volatile data at risk? (RAM, running processes)

Why Checklists Matter:

- Ensures **consistency** across investigators
 - Provides **legal defensibility**
 - Prevents **evidence contamination**
 - Creates an **audit trail**
-

Hexadecimal Notation

Number Systems Comparison:

System	Base	Digits Used	Example
Binary	2	0-1	1010
Decimal	10	0-9	10
Octal	8	0-7	12
Hexadecimal	16	0-9, A-F	A

Hexadecimal Values:

Decimal: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Hexadecimal: 0 1 2 3 4 5 6 7 8 9 A B C D E F

Conversion Examples:

Decimal Hex:

$255 \div 16 = 15$ remainder 15

$15 \div 16 = 0$ remainder 15

Reading remainders bottom-up: FF

Therefore: 255 (decimal) = FF (hex)

Hex Decimal:

$FF = (15 \times 16^1) + (15 \times 16^0)$

$= (15 \times 16) + (15 \times 1)$

$= 240 + 15$

$= 255$

Hex Binary:

Each hex digit = 4 binary bits

A F

1010 1111

Therefore: AF (hex) = 10101111 (binary)

Quick Reference - Hex to Binary:

0 = 0000 4 = 0100 8 = 1000 C = 1100

1 = 0001 5 = 0101 9 = 1001 D = 1101

2 = 0010 6 = 0110 A = 1010 E = 1110

3 = 0011 7 = 0111 B = 1011 F = 1111

Practical Bits

Understanding Bits and Bytes:

1 Bit = Single binary digit (0 or 1)

4 Bits = 1 Nibble

8 Bits = 1 Byte

16 Bits = 1 Word

32 Bits = 1 Double Word (DWORD)

64 Bits = 1 Quad Word

Storage Sizes:

1 Byte = 8 bits

1 Kilobyte = 1,024 Bytes

1 Megabyte = 1,024 KB

1 Gigabyte = 1,024 MB

1 Terabyte = 1,024 GB

1 Petabyte = 1,024 TB

Why This Matters in Forensics:

- Understanding **file sizes**
- Reading **hex dumps**
- Identifying **data patterns**
- **Slack space** calculations

Slight Diversion - ASCII

ASCII (American Standard Code for Information Interchange):

Character	Decimal	Hexadecimal	Binary
-----------	---------	-------------	--------

'A'	65	41	01000001
-----	----	----	----------

'B'	66	42	01000010
-----	----	----	----------

'a'	97	61	01100001
-----	----	----	----------

'0'	48	30	00110000
-----	----	----	----------

' '	32	20	00100000
-----	----	----	----------

Forensic Relevance: Text in files appears as recognizable hex patterns in hex editors

What is the Use of Hexadecimal?

Why Forensics Uses Hex:

1. COMPACT REPRESENTATION

Binary: 11111111 11111111

Hex: FF FF

Much easier to read!

2. FILE SIGNATURES (Magic Numbers)

Every file type has a unique hex signature

PDF: 25 50 44 46 (= %PDF)

JPEG: FF D8 FF E0

PNG: 89 50 4E 47

ZIP: 50 4B 03 04

EXE: 4D 5A (= MZ)

3. MEMORY ADDRESSES

0x7FFF5FBFF8E8

4. COLOR CODES

#FF0000 = Red

#00FF00 = Green

#0000FF = Blue

File Signature Table:

File Type	Hex Signature	ASCII
JPEG	FF D8 FF	ÿØÿ
PNG	89 50 4E 47	.PNG
PDF	25 50 44 46	%PDF
ZIP	50 4B 03 04	PK...
GIF	47 49 46 38	GIF8
EXE/DLL	4D 5A	MZ
MP3	49 44 33	ID3

Key Forensic Point: File extensions can be changed to hide files, but the **hex signature cannot lie** — forensic tools check signatures, not extensions!

Encoding and Encryption

Critical Distinction:

ENCODING $\neq$ ENCRYPTION

Encoding: For USABILITY - not for security

Encryption: For SECURITY - protects data

Encoding:

- **Purpose:** Data representation/transmission
- **Reversible:** Yes, no key needed
- **Security:** NONE
- **Examples:** ASCII, Base64, URL encoding, Unicode

Base64 Example:

"Hello" SGVsbG8=

URL Encoding:

"Hello World" Hello%20World

Encryption:

- **Purpose:** Data confidentiality
- **Reversible:** Yes, but requires a **key**
- **Security:** HIGH (if implemented correctly)
- **Examples:** AES, RSA, DES, 3DES

Types of Encryption:

SYMMETRIC ENCRYPTION

Same Key

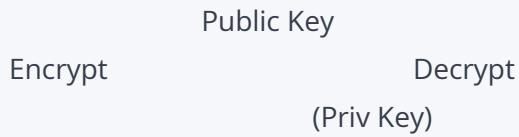
Encrypt

Decrypt

Examples: AES, DES, 3DES

Pro: Fast | Con: Key sharing problem

ASYMMETRIC ENCRYPTION



Examples: RSA, ECC

Pro: Secure key exchange | Con: Slower

Forensic Implications:

Scenario	Challenge
Encrypted disk	Cannot read without key
Encoded data	Can be decoded easily
Password protected files	May require brute force
Encrypted comms	Traffic analysis only

The Hex Editor

What is a Hex Editor?

A tool that displays file contents in **raw hexadecimal** format

Hex Editor Layout:

Offset	Hexadecimal Values	ASCII
00000000	FF D8 FF E0 00 10 4A 46 49 46	ÿØÿà..JFIF
0000000A	00 01 01 00 00 01 00 01 00 00	
00000014	FF DB 00 43 00 08 06 06 07 06	ÿÛ.C.....

Three Columns:

1. **Offset** - Position in file (hex address)
2. **Hex Values** - Raw data in hexadecimal
3. **ASCII** - Human-readable representation

Common Hex Editors:

Windows:

- HxD (Free)
- Hex Workshop
- 010 Editor

Linux:

- xxd (command line)
- GHex
- Bless

Forensic:

- FTK (built-in)
- Autopsy (built-in)
- WinHex

Forensic Uses of Hex Editor:

- **Identify file types** via magic numbers
 - **Recover deleted data** from slack space
 - **Find hidden data** (steganography)
 - **Examine metadata**
 - **Bypass renamed extensions**
-

Files

File Structure Concepts:

FILE SYSTEM COMPONENTS:

File Header	Magic bytes/signature
Metadata	Author, dates, properties
Data/Content	Actual file content
File Footer	End of file marker

Forensically Relevant File Areas:

File Slack:

Cluster Size: 4096 bytes
File Size: 3000 bytes
Slack Space: 1096 bytes May contain old data!

[===FILE DATA===][...SLACK SPACE...]
3000 bytes 1096 bytes

Types of Slack:

- **RAM Slack** - Padded with RAM data (end of sector)
- **File Slack** - Remaining space in last cluster
- **Volume Slack** - Space after last cluster
- **Partition Slack** - Space between partitions

Metadata in Files:

EXIF Data in Photos:

- Camera make/model
- GPS coordinates Location of photo!
- Date and time taken
- Camera settings

Office Documents:

- Author name
- Last modified by
- Creation date
- Revision history
- Comments

***Forensic Gold:** Metadata can **place a suspect** at a location or prove they created a document!*

Hashing

What is a Hash?

INPUT (any size) [HASH FUNCTION] FIXED SIZE OUTPUT

"Hello" [MD5] 8b1a9953c4611296a827abf8c47804d7

"Hello!" [MD5] 9c7d5b046878838da72e40cad3b08038

Completely different!

Properties of a Good Hash Function:

1. DETERMINISTIC

Same input ALWAYS = same output

2. FIXED OUTPUT SIZE

Input can be any size

Output is always the same length

3. ONE-WAY (Pre-image Resistant)

Cannot reverse hash back to original

4. AVALANCHE EFFECT

Small input change = completely different hash

5. COLLISION RESISTANT

Two different inputs should NOT produce same hash

Common Hash Algorithms:

Algorithm	Output Size	Status	Use in Forensics
MD5	128-bit (32 hex)	Weak	Legacy, still used
SHA-1	160-bit (40 hex)	Weak	Being phased out
SHA-256	256-bit (64 hex)	Strong	Current standard
SHA-512	512-bit (128 hex)	Strong	High security

Hash Example Outputs:

Input: "digital forensics"

MD5: f8c0e67c87c93d5678ea9e5d7b6f3a12
SHA-1: da39a3ee5e6b4b0d3255bfef95601890afd80709
SHA-256: 2c624232cdd221771294dfbb310acbc...

Hashing in Digital Forensics (Hashing DLs)

Why Hash in Forensics?

PURPOSE 1: EVIDENCE INTEGRITY VERIFICATION

Original Drive	Forensic Copy
Hash: ABC123	Hash: ABC123

MATCH = Exact copy proven!

PURPOSE 2: KNOWN FILE IDENTIFICATION (Hash Libraries/DLs)

KNOWN HASH DATABASES

NSRL - National Software Reference
Library (known good files)

Known malware hash databases
CSAM databases (Project VIC)

Hash Libraries (DLs) - Databases:

National Software Reference Library (NSRL):

- Contains hashes of **known legitimate software**
- Allows investigators to **filter out** OS/application files
- Saves time by focusing on **user-created content**

Project VIC / CAID:

- Database of hashes of **illegal content** (CSAM)
- Used by law enforcement worldwide
- **PhotoDNA** for image matching

The Hashing Process:

STEP 1: Hash the original evidence

Original: SHA256 = a1b2c3d4...

STEP 2: Create forensic image

dd if=/dev/sda of=evidence.img

STEP 3: Hash the forensic image

Image: SHA256 = a1b2c3d4...

STEP 4: Compare hashes

MATCH = Forensically sound copy

NO MATCH = Integrity compromised!

MD5 Hash Collisions

What is a Hash Collision?

COLLISION = Two DIFFERENT inputs produce the SAME hash output

Input A: "Document_Legitimate" MD5: abc123def456

Input B: "Document_Malicious" MD5: abc123def456

SAME HASH!

DANGEROUS!

MD5 Collision History:

1996 - Theoretical weaknesses found in MD5

2004 - Wang et al. demonstrated full MD5 collisions

2008 - Researchers created a rogue CA certificate
using MD5 collisions (Sotirov et al.)

2012 - Flame malware used MD5 collision to fake
Microsoft Windows Update signature

The Flame Malware Case:

Flame (2012) - Nation-state malware

- Used forged Microsoft certificate
- Exploited MD5 collision weakness
- Appeared as legitimate Windows update
- Considered one of most sophisticated attacks ever

Why MD5 is Still Used:

Still acceptable for:

- Evidence integrity verification (non-adversarial)
- File deduplication
- Quick integrity checks
- Legacy system compatibility

NOT acceptable for:

- Digital signatures
- Security certificates
- Password hashing
- Adversarial environments

Hash Collisions

Types of Collision Attacks:

1. COLLISION ATTACK

Find ANY two inputs with same hash

Input A $\neq$ Input B, but Hash(A) = Hash(B)

2. PRE-IMAGE ATTACK

Given hash H, find input M where Hash(M) = H

(Reversing the hash)

3. SECOND PRE-IMAGE ATTACK

Given Input A, find Input B where Hash(A) = Hash(B)

(Most dangerous for forensics!)

Collision Resistance Comparison:

Algorithm	Collision Resistance	Recommendation
MD5	BROKEN	Avoid for security
SHA-1	WEAK	Avoid for new systems
SHA-256	STRONG	Recommended
SHA-3	STRONG	Recommended

Forensic Implications of Collisions:

RISKS:

- Attacker could substitute evidence files
- Digital signatures could be forged
- Hash databases could be circumvented

MITIGATIONS:

- Use SHA-256 or stronger
- Use multiple hash algorithms together
- MD5 + SHA-1 + SHA-256 = very robust
- Chain of custody documentation

Bit Rot

What is Bit Rot?

BIT ROT (Data Degradation / Data Decay):

The gradual corruption of data stored on digital media over time, even without deliberate interference

Causes of Bit Rot:

PHYSICAL CAUSES:

- Magnetic decay (HDDs)
- Charge leakage (SSDs/Flash)
- Optical disc degradation (CDs/DVDs)
- Cosmic rays flipping bits
- Heat and humidity damage

Manufacturing defects

ELECTRONIC CAUSES:

DRAM: Needs constant refreshing

NAND Flash: Electrons leak over time

HDDs: Magnetic domains weaken

RAID: Silent data corruption

Bit Rot Timeline:

Media Type	Expected Lifespan
Magnetic Tape	10-30 years
HDD	3-5 years (active)
SSD	5-10 years
CD-R	2-10 years
DVD-R	3-20 years
M-DISC	1000+ years (claimed)

Forensic Implications:

1. EVIDENCE DEGRADATION

Old media may have corrupted sectors

2. FALSE HASH VERIFICATION

Bit rot between acquisition and examination

3. RECOVERY CHALLENGES

Partial corruption makes recovery harder

4. TIME-SENSITIVE EVIDENCE

Must acquire evidence QUICKLY

5. STORAGE OF EXHIBITS

Must store in controlled conditions

Detecting Bit Rot:

- Regular hash verification of stored images
- Error-correcting codes (ECC)
- RAID with parity (but not immune!)
- ZFS filesystem with checksumming
- Regular backup verification
- Write blockers during acquisition

Prevention Strategies:

Store at correct temperature (18-20°C)
Control humidity (30-50% RH)
Use archival-grade media
Maintain multiple copies (3-2-1 rule)
Regular integrity checking
Refresh media every 3-5 years

3-2-1 Backup Rule:

- *3 copies of data*
- *2 different media types*
- *1 offsite copy*

LAB: Encoding and Encryption

Encoding Exercises:

Base64 Encoding:

HOW BASE64 WORKS:

1. Convert text to binary (ASCII)
2. Group into 6-bit chunks
3. Map each 6-bit group to Base64 character

Base64 Alphabet:

A-Z (0-25), a-z (26-51), 0-9 (52-61), +, /

Example:


```
"Man"  01001101 01100001 01101110
        010011 010110 000101 101110
        19   22   5   46
        T    W    F    u
Result: "TWFu"
```

Command Line Tools:

```
# Linux/Mac - Base64 encoding
echo -n "Hello World" | base64
# Output: SGVsbG8gV29ybGQ=

# Base64 decoding
echo "SGVsbG8gV29ybGQ=" | base64 -d
# Output: Hello World

# URL encoding check
python3 -c "import urllib.parse; print(urllib.parse.quote('Hello World'))"
# Output: Hello%20World

# Hex dump of a file
xxd filename.txt

# MD5 hash (Linux)
md5sum filename.txt

# SHA256 hash (Linux)
sha256sum filename.txt

# Windows PowerShell hashing
Get-FileHash filename.txt -Algorithm MD5
Get-FileHash filename.txt -Algorithm SHA256
```

Hex Editor Lab Tasks:

TASK 1: File Identification

- Open unknown file in hex editor
- Check first bytes for file signature

- Compare against magic number table
- Verify against file extension

TASK 2: Metadata Extraction

- Open JPEG in hex editor
- Locate EXIF data section
- Look for GPS coordinates
- Note camera information

TASK 3: Detecting Hidden Files

- Examine file with changed extension
- Identify true file type from header
- Document findings

TASK 4: String Search

- Search hex dump for ASCII strings
 - Look for usernames, passwords, URLs
 - Document any evidence found
-